1. What is NumPy?

NumPy stands for **Numerical Python**. It is a Python library designed primarily for **efficient numerical and scientific computation**.

It is a fundamental Python library used for:

- Numerical computing
- Scientific computing
- Mathematical operations
- Linear algebra
- Statistics
- Data analysis
- Machine learning
- Signal processing
- Image processing
- Simulation

The central object of NumPy is the **ndarray (N-dimensional array)**.

A NumPy array allows us to store and process large amounts of numerical data efficiently.

Why is NumPy important?

Python itself is a general-purpose programming language. Its built-in list is flexible, but it is not specifically optimized for large-scale numerical computation.

NumPy provides a specialized numerical data structure and performs many operations using optimized compiled code.

2. Why Do We Need NumPy?

Python already provides a built-in data structure called a `list`.

For example:

numbers = [10, 20, 30, 40, 50]

So, a natural question is:

> If Python lists already store multiple values, why do we need NumPy?

The main reason is that Python lists are **general-purpose data structures**, whereas NumPy arrays are specifically optimized for **numerical computation**.

Main advantages of NumPy

1. **Fast numerical computation**
   Many NumPy operations are implemented in optimized compiled code.
2. **Memory efficiency**
   NumPy arrays store homogeneous numerical data in a compact representation.
3. **Vectorized operations**
   Operations can be applied to complete arrays without explicitly writing Python loops.
4. **Multidimensional arrays**
   NumPy naturally supports 1D, 2D, 3D, and higher-dimensional arrays.
5. **Mathematical functions**
   It provides functions such as `sqrt()`, `exp()`, `log()`, `sin()`, `cos()`, and many others.
6. **Statistical functions**
   Functions such as `mean()`, `median()`, `std()`, `min()`, and `max()` are available.
7. **Linear algebra**
   NumPy supports matrix multiplication, solving equations, eigenvalues, norms, and other operations.
8. **Foundation for scientific Python**
   Many Python libraries use NumPy arrays or concepts derived from NumPy.

In [27]:
```
# Python list
python_list = [1, 2, 3, 4, 5]

# NumPy array
numpy_array = np.array([1, 2, 3, 4, 5])

print("Python List:", python_list)
print("NumPy Array:", numpy_array)
```

```
Python List: [1, 2, 3, 4, 5]
NumPy Array: [1 2 3 4 5]
```

Basic Difference

Feature	Python List	NumPy Array
Purpose	General-purpose storage	Numerical computing
Data type	Can contain different types	Usually homogeneous
Speed	Relatively slower for numerical operations	Faster
Memory	More memory	More memory efficient
Mathematical operations	Limited	Extensive
Vectorization	Not naturally supported	Supported
Broadcasting	Not supported directly	Supported
Multidimensional data	Nested lists required	Direct support
Linear algebra	Not built-in	Built-in

NumPy is Faster than Python Lists

One of the biggest advantages of NumPy is **speed**.

Python lists are general-purpose objects. Each element of a list is a Python object, which introduces additional overhead.

NumPy arrays store numerical values in a compact, homogeneous memory layout and perform many operations using optimized compiled code.

Therefore, NumPy can perform large numerical calculations much faster than equivalent Python loops over lists.

In [28]:
```
import numpy as np
import time

size = 1_000_000

list1 = list(range(size))
list2 = list(range(size))

array1 = np.arange(size)
array2 = np.arange(size)

# Python list addition
start = time.time()

result_list = [a + b for a, b in zip(list1, list2)]

list_time = time.time() - start

# NumPy addition
start = time.time()

result_array = array1 + array2

numpy_time = time.time() - start

print("List time :", list_time)
print("NumPy time:", numpy_time)
```

```
List time : 0.11836910247802734
NumPy time: 0.0020051002502441406
```

Vectorized Operations

Vectorization means performing an operation on an entire array without explicitly writing a Python loop.

For example, suppose we want to multiply every element by 2.

Using a Python list:

In [30]:
```
numbers = [1, 2, 3, 4, 5]

result = []

for x in numbers:
    result.append(x * 2)

print(result)
```

```
[2, 4, 6, 8, 10]
```

In [31]:
```
numbers = np.array([1, 2, 3, 4, 5])

result = numbers * 2

print(result)
```

```
[ 2  4  6  8 10]
```

The NumPy version does not require an explicit Python `for` loop.

This is called **vectorization**.

Vectorization provides:

- Cleaner code
- Faster execution
- Better readability
- Efficient numerical computation

Multidimensional Arrays

NumPy naturally supports:

- 1-dimensional arrays
- 2-dimensional arrays
- 3-dimensional arrays
- N-dimensional arrays

A 2D NumPy array can represent a matrix.

For example:

```
1  2  3
4  5  6
7  8  9
```

This is much easier to manipulate using NumPy than using nested Python lists.

In [32]:
```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(matrix)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In [33]:
```
print("Dimensions:", matrix.ndim)
print("Shape:", matrix.shape)
print("Number of elements:", matrix.size)
```

```
Dimensions: 2
Shape: (3, 3)
Number of elements: 9
```

Mathematical Operations

NumPy provides many mathematical operations directly.

Examples include:

- Addition
- Subtraction
- Multiplication
- Division
- Power
- Square root
- Exponential
- Logarithm
- Trigonometric functions
- Absolute value
- Rounding

In [34]:
```
arr = np.array([1, 4, 9, 16, 25])

print("Square root:", np.sqrt(arr))
print("Square:", np.power(arr, 2))
print("Logarithm:", np.log(arr))
```

```
Square root: [1. 2. 3. 4. 5.]
Square: [  1  16  81 256 625]
Logarithm: [0.         1.38629436 2.19722458 2.77258872 3.21887582]
```

Statistical Operations

NumPy provides many statistical functions.

Important functions include:

- `np.mean()` → Mean
- `np.median()` → Median
- `np.std()` → Standard deviation
- `np.var()` → Variance
- `np.min()` → Minimum
- `np.max()` → Maximum
- `np.sum()` → Sum
- `np.prod()` → Product

In [36]:
```
data = np.array([10, 20, 30, 40, 50])

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard deviation:", np.std(data))
print("Variance:", np.var(data))
print("Minimum:", np.min(data))
print("Maximum:", np.max(data))
print("Sum:", np.sum(data))
```

```
Mean: 30.0
Median: 30.0
Standard deviation: 14.142135623730951
Variance: 200.0
Minimum: 10
Maximum: 50
Sum: 150
```

14. Linear Algebra

NumPy provides powerful tools for linear algebra.

It supports:

- Matrix multiplication
- Dot product
- Matrix transpose
- Determinant
- Inverse
- Eigenvalues
- Eigenvectors
- Solving systems of linear equations

In [37]:
```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

print("Matrix multiplication:")
print(A @ B)

print("\nTranspose:")
print(A.T)

print("\nDeterminant:")
print(np.linalg.det(A))
```

```
Matrix multiplication:
[[19 22]
 [43 50]]

Transpose:
[[1 3]
 [2 4]]

Determinant:
-2.0000000000000004
```

Creating NumPy Arrays

NumPy provides several ways to create arrays.

Common methods

- `np.array()` — create an array from a Python sequence.
- `np.zeros()` — create an array filled with zeros.
- `np.ones()` — create an array filled with ones.
- `np.arange()` — create values with a specified step.
- `np.linspace()` — create evenly spaced values over an interval.
- `np.full()` — create an array filled with a specified value.

In [5]:
```
# 1. Create from a Python list
a = np.array([1, 2, 3, 4, 5])

# 2. Array of zeros
b = np.zeros(5)

# 3. Array of ones
c = np.ones(5)

# 4. Sequence using arange
d = np.arange(0, 10, 2)

# 5. Evenly spaced values
e = np.linspace(0, 1, 5)

# 6. Array filled with a constant
f = np.full(4, 7)

print("array():", a)
print("zeros():", b)
print("ones():", c)
print("arange():", d)
print("linspace():", e)
print("full():", f)
```

```
array(): [1 2 3 4 5]
zeros(): [0. 0. 0. 0. 0.]
ones(): [1. 1. 1. 1. 1.]
arange(): [0 2 4 6 8]
linspace(): [0.   0.25 0.5  0.75 1.  ]
full(): [7 7 7 7]
```

NumPy `arange()`

`np.arange()` is used to create a NumPy array containing a sequence of numbers with a specified step size.

It is similar to Python's built-in `range()` function, but it returns a NumPy array.

For example:

```
np.arange(1, 10)
```

generates:

```
[1 2 3 4 5 6 7 8 9]
```

The important point is that the `stop` value is generally NOT included.

In [39]:
```
import numpy as np

arr = np.arange(1, 10)

print(arr)
```

```
[1 2 3 4 5 6 7 8 9]
```

Syntax of `np.arange()`

The general syntax is:

```
np.arange(start, stop, step, dtype=None)
```

Parameters:

start

The starting value of the sequence.

stop

The ending boundary of the sequence.

The `stop` value is normally excluded.

step

The difference between consecutive values.

dtype

Optional data type of the resulting array.

`arange(stop)`

If only one argument is provided, it is treated as `stop`.

The default values are:

```
start = 0
step = 1
```

Example:

```
np.arange(5)
```

is equivalent to:

```
np.arange(0, 5, 1)
```

In [40]:
```
arr = np.arange(5)

print(arr)
```

```
[0 1 2 3 4]
```

arange(start, stop)

In [41]:
```
arr = np.arange(2, 8)

print(arr)
```

```
[2 3 4 5 6 7]
```

Here:

```
start = 2
stop  = 8
step  = 1
```

Therefore:

```
2, 3, 4, 5, 6, 7
```

The value `8` is excluded.

arange() with Step

In [42]:
```
arr = np.arange(2, 10, 2)

print(arr)
```

```
[2 4 6 8]
```

Here:

```
start = 2
stop  = 10
step  = 2
```

The sequence is:

```
2 → 4 → 6 → 8 → 10
```

Since `stop = 10` is excluded:

```
[2 4 6 8]
```

Negative Step in arange()

`arange()` can also generate a decreasing sequence.

To do this, use a negative step.

In [44]:
```
arr = np.arange(10, 0, -2)

print(arr)
```

```
[10  8  6  4  2]
```

Here:

```
start = 10
stop  = 0
step  = -2
```

The sequence is:

```
10 → 8 → 6 → 4 → 2
```

The value `0` is excluded.

9. NumPy `linspace()`

`np.linspace()` is used to generate a specified number of evenly spaced values over a given interval.

The name `linspace` means:

```
Linear Space
```

For example:

```
np.linspace(0, 10, 5)
```

means:

```
Start at 0
End at 10
Generate 5 equally spaced values
```

The result is:

```
[0.  2.5  5.  7.5 10.]
```

In [45]:
```
arr = np.linspace(0, 10, 5)

print(arr)
```

```
[ 0.   2.5  5.   7.5 10. ]
```

Syntax

The basic syntax is:

```
np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=Non
e)
```

Important parameters:

start

Starting value.

stop

Ending value.

num

Number of values to generate.

The default is 50.

endpoint

Determines whether the `stop` value should be included.

Default:

```
endpoint=True
```

retstep

If `True` , NumPy also returns the spacing between consecutive values.

dtype

Optional data type of the output.

In [46]:
```
arr = np.linspace(1, 10, 5)

print(arr)
```

```
[ 1.    3.25  5.5   7.75 10.  ]
```

Here:

```
start = 1
stop  = 10
num   = 5
```

We want exactly 5 values.

The interval is:

```
1 to 10
```

The distance between adjacent values is:

```
(10 - 1) / (5 - 1)

= 9 / 4

= 2.25
```

Therefore:

```
1
1 + 2.25 = 3.25
3.25 + 2.25 = 5.50
5.50 + 2.25 = 7.75
7.75 + 2.25 = 10
```

Result:

```
[1.00, 3.25, 5.50, 7.75, 10.00]
```

In [ ]:
```

```

Quick Comparison

Feature	`np.arange()`	`np.linspace()`
Main idea	Specify step	Specify number of values
Syntax	`start, stop, step`	`start, stop, num`
Stop included?	Usually no	Yes, by default
Best for	Known step size	Known number of points
Floating-point use	Can have precision issues	Generally preferable

Feature	`np.arange()`	`np.linspace()`
Number of values	Determined by range and step	Explicitly specified
Common use	Integer sequences	Mathematical intervals

6. Important Properties of a NumPy Array

Every NumPy array has several important properties.

`ndim`

Number of dimensions.

`shape`

Number of elements along each dimension.

`size`

Total number of elements.

`dtype`

Data type of the elements.

For example, a matrix with 2 rows and 3 columns has:

- `ndim = 2`
- `shape = (2, 3)`
- `size = 6`

In [ ]:
```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print("Matrix:")
print(matrix)

print("\nNumber of dimensions:", matrix.ndim)
print("Shape:", matrix.shape)
print("Total elements:", matrix.size)
print("Data type:", matrix.dtype)
```

13. Reshaping Arrays

`reshape()` changes the dimensions of an array while keeping the same number of elements.

For example, 12 values can be reshaped into:

- 3 × 4
- 2 × 6
- 4 × 3
- 1 × 12

The total number of elements must remain unchanged

In [ ]:
```
data = np.arange(1, 13)

print("Original array:")
print(data)

matrix = data.reshape(3, 4)

print("\n3 x 4 matrix:")
print(matrix)

print("\nTranspose:")
print(matrix.T)
```

Slicing in a 1D NumPy Array

Slicing is used to extract a portion of an array.

The general syntax is:

```
array[start : stop : step]
```

Where:

- `start` → index from where slicing starts
- `stop` → index where slicing stops (NOT included)
- `step` → interval between elements

If a value is omitted, NumPy uses a default:

- `start` → beginning of the array
- `stop` → end of the array
- `step` → 1

In [47]:
```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70])

print("Array:", arr)

print("arr[1:5]:", arr[1:5])
print("arr[:4]:", arr[:4])
print("arr[3:]:", arr[3:])
```

```
Array: [10 20 30 40 50 60 70]
arr[1:5]: [20 30 40 50]
arr[:4]: [10 20 30 40]
arr[3:]: [40 50 60 70]
```

Slicing with Step and Negative Index

We can specify a step:

```
arr[start : stop : step]
```

Example:

```
arr[0:7:2]
```

means:

- Start at index `0`
- Stop before index `7`
- Take every 2nd element

Negative indices can be used to access elements from the end.

For example:

```
arr[-1] → last element
arr[-2] → second-last element
```

We can also reverse an array using:

```
arr[::-1]
```

In [48]:
```
print("Every 2nd element:", arr[0:7:2])
print("Every 3rd element:", arr[0:7:3])

print("Last 3 elements:", arr[-3:])
print("Reverse array:", arr[::-1])
```

```
Every 2nd element: [10 30 50 70]
Every 3rd element: [10 40 70]
Last 3 elements: [50 60 70]
Reverse array: [70 60 50 40 30 20 10]
```

Slicing in a 2D NumPy Array

A 2D array has two dimensions:

```
rows × columns
```

For example:

```
    Columns
    0   1   2   3
0  10  20  30  40
1  50  60  70  80
2  90 100 110 120
```

The general syntax for slicing a 2D array is:

```
array[row_start : row_stop : row_step,
      col_start : col_stop : col_step]
```

The part before the comma controls **rows**. The part after the comma controls **columns**.

Remember:

```
start → included
stop  → excluded
```

In [49]:
```
import numpy as np

arr = np.array([
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
])

print(arr)

# First two rows
print("\nFirst two rows:")
print(arr[:2, :])

# First three columns
print("\nFirst three columns:")
print(arr[:, :3])

# Rows 1 and 2, columns 1 and 2
print("\nSelected portion:")
print(arr[1:3, 1:3])
```

```
[[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]

First two rows:
[[10 20 30 40]
 [50 60 70 80]]

First three columns:
[[ 10  20  30]
 [ 50  60  70]
 [ 90 100 110]]

Selected portion:
[[ 60  70]
 [100 110]]
```

Important 2D Slicing Examples

Consider:

```
arr =
[[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]
```

1. Select one row

```
arr[1, :]
```

Output:

```
[50 60 70 80]
```

2. Select one column

```
arr[:, 2]
```

Output:

```
[30 70 110]
```

3. Select rows 0 and 1

```
arr[0:2, :]
```

4. Select columns 1 and 2

```
arr[:, 1:3]
```

5. Select a rectangular portion

```
arr[0:2, 1:3]
```

Output:

```
[[20 30]
 [60 70]]
```

14. Application of NumPy in Scientific Computing

Scientific computing deals heavily with numerical data, mathematical models, simulations, and large arrays.

NumPy is used in:

14.1 Physics

Physical quantities such as position, velocity, acceleration, temperature, pressure, and energy can be represented as arrays.

14.2 Engineering

Engineers use numerical arrays for simulations, measurements, signal processing, control systems, and mathematical modelling.

14.3 Statistics

NumPy supports mean, variance, standard deviation, percentiles, random sampling, and other numerical operations.

14.4 Numerical simulation

Scientists can generate large collections of values and repeatedly apply mathematical equations.

14.5 Scientific data processing

Experimental measurements and sensor data can be stored and processed efficiently.

14.6 Linear algebra

Many scientific problems can be represented using vectors and matrices.

15. Scientific Computing Example — Motion

Consider the equation:

$$s(t) = ut + \frac{1}{2}at^2$$

where:

- s = displacement
- u = initial velocity
- a = acceleration
- t = time

Instead of calculating the displacement separately for every time value, NumPy can calculate all values at once.

In [ ]:
```
# Motion example

u = 5       # initial velocity in m/s
a = 2       # acceleration in m/s^2

t = np.array([0, 1, 2, 3, 4, 5])

s = u * t + 0.5 * a * t**2

print("Time (s):", t)
print("Displacement (m):", s)
```

16. Scientific Computing Example — Temperature Analysis

Suppose a laboratory records temperature measurements.

NumPy can quickly calculate:

- Average temperature
- Minimum temperature

- Maximum temperature
- Standard deviation

In [ ]:
```
temperature = np.array([
    24.2, 25.1, 26.4, 27.0,
    28.3, 27.6, 26.1
])

print("Average:", np.mean(temperature))
print("Minimum:", np.min(temperature))
print("Maximum:", np.max(temperature))
print("Standard deviation:", np.std(temperature))
```

18. Linear Algebra with NumPy

Linear algebra is one of NumPy's most important applications.

It is used in:

- Physics
- Engineering
- Computer graphics
- Machine learning
- Optimization
- Computer vision

NumPy provides the `np.linalg` module for linear algebra.

Common operations include:

- Matrix multiplication
- Matrix inverse
- Solving linear equations
- Eigenvalues and eigenvectors
- Matrix norms

In [ ]:
```
A = np.array([
    [2, 1],
    [1, 3]
])

B = np.array([
    [1, 2],
    [3, 4]
])

print("A:")
print(A)

print("\nB:")
print(B)

print("\nMatrix multiplication A @ B:")
print(A @ B)

print("\nTranspose of A:")
print(A.T)
```

19. Solving Linear Equations

Consider:

$$2x + y = 5$$

$$x + 3y = 6$$

This can be written as:

$$A\mathbf{x} = \mathbf{b}$$

where:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

and:

$$\mathbf{b} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

NumPy can solve this system using `np.linalg.solve()` .

In [ ]:
```
A = np.array([
    [2, 1],
    [1, 3]
])

b = np.array([5, 6])

solution = np.linalg.solve(A, b)

print("Solution [x, y]:", solution)

x, y = solution

print("\nVerification:")
print("2x + y =", 2*x + y)
print("x + 3y =", x + 3*y)
```

20. Applications of NumPy in Computer Science

NumPy is used in many areas of computer science.

20.1 Machine Learning

Machine-learning algorithms use vectors and matrices for:

- Feature representation
- Model parameters
- Weight matrices
- Numerical transformations
- Dataset processing

NumPy is especially useful for learning and implementing the mathematical foundations of machine learning.

20.2 Data Science

NumPy provides efficient numerical operations and is an important part of the Python scientific-computing ecosystem.

20.3 Computer Vision

Images can be represented as NumPy arrays.

A grayscale image can be represented as:

$$H \times W$$

and an RGB image commonly as:

$$H \times W \times 3$$

where `H` is height and `W` is width.

20.4 Computer Graphics

Vectors and matrices are used for:

- Rotation
- Scaling
- Translation
- Projection
- Coordinate transformations

NumPy can implement these mathematical operations.

20.5 Signal Processing

Audio, sensor, and other signals are numerical sequences. NumPy provides efficient array operations useful in signal-processing workflows.

20.6 Artificial Intelligence

Many AI algorithms require vector and matrix operations. NumPy is useful for understanding and implementing these numerical operations, although deep-learning frameworks commonly use specialized tensor libraries for GPU acceleration.

20.7 Numerical Algorithms

NumPy can be used to implement:

- Numerical integration
- Interpolation
- Simulation
- Optimization foundations
- Statistical algorithms

21. Computer Vision Example — Image as an Array

A grayscale image can be represented as a 2D NumPy array.

Each element represents the intensity of a pixel.

For a typical 8-bit grayscale image:

- `0` represents black.
- `255` represents white.
- Values between them represent different shades of gray.

In [ ]:
```
# A small grayscale image

image = np.array([
    [0, 50, 100],
    [150, 200, 255],
    [100, 150, 200]
], dtype=np.uint8)

print("Image:")
print(image)

print("\nShape:", image.shape)
print("Number of pixels:", image.size)
print("Maximum intensity:", image.max())
```

22. Computer Graphics Example — Rotation

A 2D point can be represented as:

$$p = \begin{bmatrix} x \\ y \end{bmatrix}$$

A rotation matrix is:

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

The rotated point is:

$$p' = Rp$$

NumPy makes this matrix operation straightforward.

In [ ]:
```
theta = np.deg2rad(90)

R = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta),  np.cos(theta)]
])

point = np.array([1, 0])

rotated_point = R @ point

print("Rotation matrix:")
print(R)

print("\nOriginal point:", point)
print("Rotated point:", np.round(rotated_point, 10))
```

24. Important NumPy Functions to Remember

Function	Purpose
`np.array()`	Create an array
`np.zeros()`	Create an array of zeros
`np.ones()`	Create an array of ones
`np.arange()`	Generate a numerical sequence
`np.linspace()`	Generate evenly spaced values
`np.reshape()`	Change array shape
`np.sum()`	Sum elements
`np.mean()`	Calculate mean
`np.median()`	Calculate median
`np.std()`	Calculate standard deviation
`np.min()`	Find minimum
`np.max()`	Find maximum
`np.sqrt()`	Square root
`np.exp()`	Exponential
`np.dot()`	Dot product
`np.linalg.solve()`	Solve linear equations
`np.random.default_rng()`	Create random-number generator

Vector and Vector Operations Using NumPy

Overview

Vectors are fundamental mathematical objects used in physics, engineering, mathematics, computer graphics, machine learning, robotics, and scientific computing.

NumPy provides efficient and convenient tools for representing and manipulating vectors using `numpy.ndarray` .

In this notebook, we will study:

1. Vectors
2. Vector addition and subtraction
3. Scalar multiplication
4. Magnitude of a vector
5. Unit vectors
6. Dot product
7. Cross product
8. Scalar triple product
9. Vector triple product
10. Dyadic product
11. Comparison of different vector products

1. Introduction to Vectors

A **vector** is a mathematical quantity that has both **magnitude and direction**.

A three-dimensional vector can be represented as:

$$\mathbf{a} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$$

For example:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

In NumPy, a vector is commonly represented using a one-dimensional NumPy array:

```
a = np.array([2, 3, 4])
```

In [7]:
```
import numpy as np

# Creating a vector
a = np.array([2, 3, 4])

print("Vector a:", a)
print("Type:", type(a))
print("Shape:", a.shape)
```

```
Vector a: [2 3 4]
Type: <class 'numpy.ndarray'>
Shape: (3,)
```

2. Components of a Vector

Consider:

$$\mathbf{a} = (2, 3, 4)$$

The components are:

- $a_x = 2$
- $a_y = 3$
- $a_z = 4$

Individual components can be accessed using NumPy indexing.

In [8]:
```
a = np.array([2, 3, 4])

print("x-component:", a[0])
print("y-component:", a[1])
print("z-component:", a[2])
```

```
x-component: 2
y-component: 3
z-component: 4
```

3. Vector Addition

If:

$$\mathbf{a} = (a_1, a_2, a_3)$$

and

$$\mathbf{b} = (b_1, b_2, b_3)$$

then:

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1,\ a_2 + b_2,\ a_3 + b_3)$$

For example:

$$\mathbf{a} = (1, 2, 3)$$

$$\mathbf{b} = (4, 5, 6)$$

Therefore:

$$\mathbf{a} + \mathbf{b} = (5, 7, 9)$$

In [9]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = a + b

print("a + b =", result)
```

```
a + b = [5 7 9]
```

4. Vector Subtraction

Vector subtraction is performed component by component:

$$\mathbf{a} - \mathbf{b} = (a_1 - b_1,\ a_2 - b_2,\ a_3 - b_3)$$

For:

$$\mathbf{a} = (1, 2, 3)$$

and:

$$\mathbf{b} = (4, 5, 6)$$

we obtain:

$$\mathbf{a} - \mathbf{b} = (-3, -3, -3)$$

In [10]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = a - b

print("a - b =", result)
```

```
a - b = [-3 -3 -3]
```

5. Scalar Multiplication

A vector can be multiplied by a scalar.

If:

$$\mathbf{a} = (1, 2, 3)$$

then:

$$3\mathbf{a} = (3, 6, 9)$$

In NumPy, scalar multiplication is performed using the * operator.

In [11]:
```
a = np.array([1, 2, 3])

result = 3 * a

print("3a =", result)
```

```
3a = [3 6 9]
```

6. Magnitude of a Vector

The magnitude or length of a vector is:

$$|\mathbf{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

For:

$$\mathbf{a} = (3, 4, 0)$$

we have:

$$|\mathbf{a}| = \sqrt{3^2 + 4^2 + 0^2}$$

$$= \sqrt{25}$$

$$= 5$$

NumPy provides:

```
np.linalg.norm()
```

In [12]:
```
a = np.array([3, 4, 0])

magnitude = np.linalg.norm(a)

print("Vector:", a)
print("Magnitude:", magnitude)
```

```
Vector: [3 4 0]
Magnitude: 5.0
```

7. Unit Vector

A **unit vector** is a vector whose magnitude is equal to 1.

The unit vector corresponding to $\mathbf{a}$ is:

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{|\mathbf{a}|}$$

For:

$$\mathbf{a} = (3, 4, 0)$$

we have:

$$|\mathbf{a}| = 5$$

Therefore:

$$\hat{\mathbf{a}} = \left(\frac{3}{5}, \frac{4}{5}, 0\right)$$

or:

$$\hat{\mathbf{a}} = (0.6, 0.8, 0)$$

In [13]:
```
a = np.array([3, 4, 0])

magnitude = np.linalg.norm(a)
unit_vector = a / magnitude

print("Original vector:", a)
print("Magnitude:", magnitude)
print("Unit vector:", unit_vector)
print("Magnitude of unit vector:",
      np.linalg.norm(unit_vector))
```

```
Original vector: [3 4 0]
Magnitude: 5.0
Unit vector: [0.6 0.8 0. ]
Magnitude of unit vector: 1.0
```

8. Dot Product

The **dot product**, also called the **scalar product**, takes two vectors and produces a scalar.

For:

$$\mathbf{a} = (a_1, a_2, a_3)$$

and:

$$\mathbf{b} = (b_1, b_2, b_3)$$

the dot product is:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

Example

Let:

$$\mathbf{a} = (1, 2, 3)$$

and:

$$\mathbf{b} = (4, 5, 6)$$

Then:

$$\mathbf{a} \cdot \mathbf{b} = 1(4) + 2(5) + 3(6)$$

$$= 4 + 10 + 18$$

$$= 32$$

Therefore:

$$\boxed{\mathbf{a} \cdot \mathbf{b} = 32}$$

In [14]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

dot_product = np.dot(a, b)

print("a =", a)
print("b =", b)
print("Dot product =", dot_product)
```

```
a = [1 2 3]
b = [4 5 6]
Dot product = 32
```

9. Dot Product Using the @ Operator

NumPy also supports the matrix multiplication operator `@` .

For one-dimensional arrays,:

```
a @ b
```

In [16]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = a @ b

print("Dot product:", result)
```

```
Dot product: 32
```

11. Geometrical Meaning of Dot Product

The dot product can also be expressed as:

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta$$

where θ is the angle between the two vectors.

Therefore:

$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}||\mathbf{b}|}$$

and:

$$\theta = \cos^{-1}\left(\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}||\mathbf{b}|}\right)$$

In [17]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

dot_product = np.dot(a, b)

cos_theta = dot_product / (
    np.linalg.norm(a) * np.linalg.norm(b)
)

theta = np.arccos(cos_theta)

print("cos(theta) =", cos_theta)
print("Angle in radians =", theta)
print("Angle in degrees =", np.degrees(theta))
```

```
cos(theta) = 0.9746318461970762
Angle in radians = 0.2257261285527342
Angle in degrees = 12.933154491899135
```

13. Cross Product

The **cross product**, also called the **vector product**, takes two three-dimensional vectors and produces another vector.

For:

$$\mathbf{a} = (a_1, a_2, a_3)$$

and:

$$\mathbf{b} = (b_1, b_2, b_3)$$

the cross product is:

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

Expanding:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

Unlike the dot product, the result is a **vector**.

In [18]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

cross_product = np.cross(a, b)

print("a =", a)
print("b =", b)
print("Cross product =", cross_product)
```

```
a = [1 2 3]
b = [4 5 6]
Cross product = [-3  6 -3]
```

14. Geometrical Meaning of Cross Product

The magnitude of the cross product is:

$$|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}||\mathbf{b}| \sin\theta$$

where θ is the angle between the vectors.

The magnitude represents the **area of the parallelogram** formed by the two vectors.

Therefore:

$$\boxed{A = |\mathbf{a} \times \mathbf{b}|}$$

The area of the triangle formed by the vectors is:

$$A = \frac{1}{2}|\mathbf{a} \times \mathbf{b}|$$

16. Triple Product

There are two important types of triple products:

1. **Scalar Triple Product**
2. **Vector Triple Product**

They have different expressions and produce different types of results.

Triple Product	Expression	Result
Scalar Triple Product	$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$	Scalar
Vector Triple Product	$\mathbf{a} \times (\mathbf{b} \times \mathbf{c})$	Vector

17. Scalar Triple Product

The scalar triple product is:

$$\boxed{\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})}$$

The operation is performed from the inside:

Step 1

Calculate:

$$\mathbf{b} \times \mathbf{c}$$

Step 2

Calculate:

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$$

The final result is a **scalar**.

In [21]:
```
a = np.array([1, 0, 0])
b = np.array([0, 2, 0])
c = np.array([0, 0, 3])

cross_bc = np.cross(b, c)
scalar_triple = np.dot(a, cross_bc)

print("b × c =", cross_bc)
print("a · (b × c) =", scalar_triple)
```

```
b × c = [6 0 0]
a · (b × c) = 6
```

18. Geometrical Meaning of Scalar Triple Product

The absolute value of the scalar triple product represents the volume of the parallelepiped formed by three vectors.

$$\boxed{V = |\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})|}$$

For:

$$\mathbf{a} = (1, 0, 0)$$

$$\mathbf{b} = (0, 2, 0)$$

$$\mathbf{c} = (0, 0, 3)$$

the volume is:

$$V = 1 \times 2 \times 3 = 6$$

19. Scalar Triple Product Using Determinant

The scalar triple product can also be calculated using a determinant:

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \begin{vmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{vmatrix}$$

NumPy provides:

```
np.linalg.det()
```

In [23]:
```
matrix = np.array([
    [1, 0, 0],
    [0, 2, 0],
    [0, 0, 3]
])

determinant = np.linalg.det(matrix)

print("Determinant:", determinant)
```

```
Determinant: 6.0
```

20. Vector Triple Product

The vector triple product is:

$$\boxed{\mathbf{a} \times (\mathbf{b} \times \mathbf{c})}$$

Unlike the scalar triple product, the result is a **vector**.

An important identity is:

$$\boxed{\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a} \cdot \mathbf{c}) - \mathbf{c}(\mathbf{a} \cdot \mathbf{b})}$$

This is commonly called the **BAC-CAB identity**.

In [24]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.array([7, 8, 9])

vector_triple = np.cross(a, np.cross(b, c))

print("a × (b × c) =", vector_triple)
```

```
a × (b × c) = [-24  -6  12]
```

21. Verifying the BAC-CAB Identity

We can verify:

$$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a} \cdot \mathbf{c}) - \mathbf{c}(\mathbf{a} \cdot \mathbf{b})$$

using NumPy.

In [25]:
```
left_side = np.cross(a, np.cross(b, c))

right_side = (
    b * np.dot(a, c)
    - c * np.dot(a, b)
)

print("Left side :", left_side)
print("Right side:", right_side)

print("Are they equal?",
      np.allclose(left_side, right_side))
```

```
Left side : [-24  -6  12]
Right side: [-24  -6  12]
Are they equal? True
```

22. Dyadic Product

The **dyadic product** is also known as the **outer product** of two vectors.

For vectors:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

and:

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

the dyadic product is:

$$\mathbf{a} \otimes \mathbf{b} = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_3 b_3 \end{bmatrix}$$

The result is a **matrix**, which can be viewed as a second-order tensor.

In [26]:
```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

dyadic = np.outer(a, b)

print("Dyadic product:")
print(dyadic)
```

```
Dyadic product:
[[ 4  5  6]
 [ 8 10 12]
 [12 15 18]]
```

23. Understanding the Dyadic Product

Consider:

$$\mathbf{a} = (1, 2, 3)$$

and:

$$\mathbf{b} = (4, 5, 6)$$

Every element of $\mathbf{a}$ is multiplied by every element of $\mathbf{b}$:

$$\mathbf{a} \otimes \mathbf{b} = \begin{bmatrix} 1(4) & 1(5) & 1(6) \\ 2(4) & 2(5) & 2(6) \\ 3(4) & 3(5) & 3(6) \end{bmatrix}$$

Therefore:

$$= \begin{bmatrix} 4 & 5 & 6 \\ 8 & 10 & 12 \\ 12 & 15 & 18 \end{bmatrix}$$

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [20]:
```
import numpy as np

# Define vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.array([7, 8, 9])

# Basic vector operations
addition = a + b
subtraction = a - b
scalar_multiplication = 2 * a

# Magnitude
magnitude = np.linalg.norm(a)

# Unit vector
unit_vector = a / magnitude

# Dot product
dot_product = np.dot(a, b)

# Cross product
cross_product = np.cross(a, b)

# Scalar triple product
scalar_triple_product = np.dot(
    a,
    np.cross(b, c)
)

# Vector triple product
vector_triple_product = np.cross(
    a,
    np.cross(b, c)
)

# Dyadic product
dyadic_product = np.outer(a, b)

# Display results
print("Vector a:", a)
print("Vector b:", b)
print("Vector c:", c)

print("\nVector Addition:")
print(addition)

print("\nVector Subtraction:")
print(subtraction)

print("\nScalar Multiplication:")
print(scalar_multiplication)

print("\nMagnitude of a:")
print(magnitude)

print("\nUnit Vector of a:")
```

```
print(unit_vector)

print("\nDot Product:")
print(dot_product)

print("\nCross Product:")
print(cross_product)

print("\nScalar Triple Product:")
print(scalar_triple_product)

print("\nVector Triple Product:")
print(vector_triple_product)

print("\nDyadic Product:")
print(dyadic_product)
```

```
Vector a: [1 2 3]
Vector b: [4 5 6]
Vector c: [7 8 9]

Vector Addition:
[5 7 9]

Vector Subtraction:
[-3 -3 -3]

Scalar Multiplication:
[2 4 6]

Magnitude of a:
3.7416573867739413

Unit Vector of a:
[0.26726124 0.53452248 0.80178373]

Dot Product:
32

Cross Product:
[-3  6 -3]

Scalar Triple Product:
0

Vector Triple Product:
[-24  -6  12]

Dyadic Product:
[[ 4  5  6]
 [ 8 10 12]
 [12 15 18]]
```

30. Final Conceptual Understanding

The most important distinction to remember is the type of output produced by each operation.

Dot Product

$$\boxed{\mathbf{a} \cdot \mathbf{b} \rightarrow \text{Scalar}}$$

It measures the degree of alignment between two vectors.

Cross Product

$$\boxed{\mathbf{a} \times \mathbf{b} \rightarrow \text{Vector}}$$

It produces a vector perpendicular to the two input vectors.

Scalar Triple Product

$$\boxed{\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) \rightarrow \text{Scalar}}$$

Its absolute value represents the volume of a parallelepiped.

Vector Triple Product

$$\boxed{\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) \rightarrow \text{Vector}}$$

It follows the BAC-CAB identity:

$$\boxed{\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a} \cdot \mathbf{c}) - \mathbf{c}(\mathbf{a} \cdot \mathbf{b})}$$

Dyadic Product

$$\boxed{\mathbf{a} \otimes \mathbf{b} \rightarrow \text{Matrix/Tensor}}$$

It multiplies every component of one vector by every component of the other vector.

In NumPy:

```
np.outer(a, b)
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

27. Important Summary

Operation	Mathematical Form	NumPy Function	Result
Vector addition	$\mathbf{a} + \mathbf{b}$	`a + b`	Vector
Vector subtraction	$\mathbf{a} - \mathbf{b}$	`a - b`	Vector
Scalar multiplication	$k\mathbf{a}$	`k * a`	Vector
Magnitude	$\|\mathbf{a}\|$	`np.linalg.norm(a)`	Scalar
Dot product	$\mathbf{a} \cdot \mathbf{b}$	`np.dot(a,b)`	Scalar
Cross product	$\mathbf{a} \times \mathbf{b}$	`np.cross(a,b)`	Vector
Scalar triple product	$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$	`np.dot(a,np.cross(b,c))`	Scalar
Vector triple product	$\mathbf{a} \times (\mathbf{b} \times \mathbf{c})$	`np.cross(a,np.cross(b,c))`	Vector
Dyadic product	$\mathbf{a} \otimes \mathbf{b}$	`np.outer(a,b)`	Matrix/Tensor

In [ ]: